

Risiko ID	Beschreibung	Wahrscheinlichkeit (1–5)	Schadenshöhe (1–5)	Risiko-Score	Minimierungs-Strategie	Indikatoren	Notfallplan	Status	Verantwortlicher	Letzte Aktualisierung	
R01	Zeitmanagement im Semester: Neben anderen Modulen, Prüfungen und Abgaben bleibt weniger Entwicklungszeit als geplant.	5	4	20	MVP strikt priorisieren, Aufgaben klein schneiden, wöchentliche Abstimmung, früh mit Kernfunktionen anfangen	Tickets bleiben lange offen, Meilensteine verschieben sich, mehrere Aufgaben werden gleichzeitig halb fertig	Nicht-MVP-Funktionen verschieben, Funktionsumfang reduzieren, nur Kern-Use-Cases fertigstellen	Unter Kontrolle	Leon	09.06.2026	Retro: Alle Abgaben und MVP rechtzeitig geschafft –
R02	Anforderungen ändern sich während der Bearbeitung, weil Ideen im Team weiterreifen oder nach Feedback angepasst werden.	4	4	16	Anforderungen im SRS festhalten, Änderungen im Team kurz abstimmen, MVP und Nice-to-have trennen	häufige Änderungen an User Stories, Diskussionen über Prioritäten, widersprüchliche Beschreibungen	Scope Freeze für aktuelle Iteration, neue Ideen ins Backlog statt sofort umsetzen	Eingetreten	Leon	09.06.2026	Retro: Eingetreten – unklare Definition of
R03	Technisches Risiko: Das Team hat noch zu wenig Erfahrung mit Teilen des Stacks (z. B. Next.js, Prisma, Auth, Rollenlogik).	4	4	16	Früher Prototyp, Spikes für unbekannte Themen, Tutorials/Doku nutzen, schwierige Themen früh testen	lange Blocker bei Setup/Implementierung, häufige Rückfragen, viele Fehlversuche	Vereinfachte technische Lösung wählen, weniger komplexe Features umsetzen, Hilfe im Kurs/Tutorium einholen	Offen	Mykyta	20.04.2026	Retro: Kein akuter Befund – keine Stack-Blocker
R04	Technisches Risiko: Fehler in der Berechnung von Ausgaben, Anteilen und Salden führen zu fachlich falschen Ergebnissen.	3	5	15	Rechenlogik zentral kapseln, Unit-Tests mit Beispielergebnissen, Randfälle für Rundung und Aufteilung definieren	unterschiedliche Ergebnisse bei gleichen Eingaben, falsche Salden, Testfälle schlagen fehl	Berechnung vorübergehend auf einfache Gleichverteilung beschränken, fehlerhafte Funktion markieren und manuell prüfen	Offen	Denis	09.06.2026	Retro: Pair Programming bei der Budget-Aufteilung half;
R05	Technisches Risiko: Authentifizierung, WG-Beitritt und Rollen (Admin/Bewohner) funktionieren nicht korrekt oder unsicher.	3	5	15	Login und Rollenprüfung früh implementieren, serverseitige Rechteprüfung, Testfälle für UC1–UC3	Nutzer können WG nicht beitreten, falsche Rechte, Einladungslinks funktionieren nicht	Rollenlogik vereinfachen, problematische Admin-Funktionen sperren, nur minimale Rechteverwaltung für MVP aktivieren	Offen	Denis	20.04.2026	Retro: Kein Befund zu Auth/Rollen.
R06	Teamkoordination: Aufgabenteilung ist unklar oder Wissen liegt nur bei einzelnen Personen.	3	4	12	Tickets klar verteilen, kurze Dailys/Weeklys, Git sauber nutzen, Wissen dokumentieren	eine Person macht kritische Bereiche allein, Merge-Konflikte, Unklarheit über Zuständigkeiten	Aufgaben neu verteilen, Pair Programming, Übergabedokumentation erstellen	Teilweise eingetreten	Leon	09.06.2026	Retro: Kommunikation lief gut, aber GitHub-Board
R07	Technisches Risiko: Datenbank, Schema oder Migrationen verursachen Probleme, wodurch Daten inkonsistent werden oder Features blockiert sind.	3	4	12	Datenmodell früh festlegen, einfache Struktur wählen, Migrationen lokal testen, Seed-Daten verwenden	Migrationsfehler, kaputte Testdaten, wiederholte DB-Probleme	Auf letzte stabile Version zurückgehen, Schema vereinfachen, Daten manuell bereinigen	Offen	Denis	20.04.2026	Retro: Kein Befund zu DB/Migrationen.
R08	Technische Schulden entstehen, weil aus Zeitdruck schnelle Workarounds eingebaut werden.	4	3	12	Coding-Standards, Reviews, einfache Architektur, bewusst nur wenige Kernfeatures sauber umsetzen	duplizierter Code, unklare Komponentenstruktur, steigende Bugzahl	Refactoring vor Abgabe der finalen Version, kritische Stellen priorisiert bereinigen	Beobachten	Leon	09.06.2026	Retro: Hinweise auf unklare Architektur / unterschiedliche
R09	UI/Usability-Probleme: Die Webapp ist auf Handy oder im Browser nicht gut nutzbar.	3	3	9	Früh responsive testen, einfache Seitenstruktur, Kernabläufe mit Kommiliton*innen testen	Layout bricht, Buttons schlecht bedienbar, unklare Navigation	Mobile Darstellung vereinfachen, problematische Komponenten entfernen oder vereinfachen	Offen	Kim	20.04.2026	Retro: Kein Befund zu UI/Usability.
R10	Git-/Merge-Probleme verzögern die Entwicklung oder überschreiben Arbeitsergebnisse.	3	3	9	Branching-Regeln, kleine Commits, Pull Requests, häufiges Pullen und Mergen	Merge-Konflikte häufen sich, Features blockieren sich gegenseitig, versehentliche Überschreibungen	Konflikte gemeinsam lösen, letzten stabilen Stand wiederherstellen, kurzfristig Paararbeit am selben Feature	Offen	Denis	09.06.2026	Retro: Keine akuten Merge-Probleme genannt;
R11	Datenschutz- und Sicherheitsfehler: Personenbezogene Daten oder WG-interne Informationen sind durch fehlerhafte Berechtigungen, unsichere Konfiguration oder ungeschützte Einladungslinks sichtbar.	3	5	15	Nur notwendige Daten speichern, serverseitige Rechteprüfung, sichere Invite-Codes, Testfälle für Berechtigungen und Zugriffsschutz	Nutzer sehen Daten fremder WGs, Einladungen sind leicht erratbar, sensible Daten erscheinen in Logs oder URLs	Betroffene Funktion sofort deaktivieren, fehlerhafte Einladungen ungültig machen, Rechteprüfung nachbessern und erneut testen	Offen	Yaroslav	20.04.2026	Retro: Kein Befund zu Datenschutz/Sicherheit.
R12	Fehlende oder zu späte Tests: Wichtige Funktionen werden erst kurz vor der Abgabe geprüft, sodass Fehler in Kern-Use-Cases zu spät auffallen.	4	4	16	Früh einfache Unit-Tests und manuelle Tests für MVP-Use-Cases festlegen, Testfälle aus User Stories und Use Cases ableiten, vor jedem Meilenstein kurz testen	Viele Bugs kurz vor Abgabe, unklare Testabdeckung, Features funktionieren nur in Einzelfällen	Testfokus auf Kernfunktionen legen, Nice-to-have-Features zurückstellen, bekannte Fehler dokumentieren und kritische Bugs priorisiert beheben	Eingetreten	Kim	09.06.2026	Retro: Eingetreten – kaum automatisierte